

EKF Tracking of ESCNN Weights: Unsupervised Soft-Syndrome (**ekf**) and Supervised (**ekfi**) Measurements

1 Unsupervised EKF via soft-syndrome measurements (**ekf**)

1.1 State-space model

The ESCNN weights are treated as a latent state $\boldsymbol{\theta}_i \in \mathbb{R}^d$ that drifts slowly from frame to frame, and is tracked using only the channel code’s parity structure — no ground-truth bits are needed.

State evolution

$$\boldsymbol{\theta}_i = f(\boldsymbol{\theta}_{i-1}) + \mathbf{w}_i, \quad Q = \text{Cov}(\mathbf{w}_i) \quad (1)$$

In the simplest (and most common) case f is the identity, i.e. the weights follow a random walk and the process noise Q controls how fast tracking is allowed to move.

Choice of f . Two natural dynamics models:

- **Random walk:** $f(\boldsymbol{\theta}) = \boldsymbol{\theta}$, so $F_i = I$. The weight estimate has no pull-back term — under this model, whatever the filter drifts to is held purely by the balance of Q and R , with no anchor to a known-good solution.
- **First-order Markov / AR(1):** $f(\boldsymbol{\theta}) = \alpha \boldsymbol{\theta} + (1 - \alpha) \boldsymbol{\theta}_{\text{pretrained}}$, $0 < \alpha \leq 1$, so $F_i = \alpha I$. The weights are mean-reverting toward the pretrained solution $\boldsymbol{\theta}_{\text{pretrained}}$ at rate $1 - \alpha$ per frame; $\alpha = 1$ recovers the random walk exactly. This adds an implicit regularizer that keeps the filter anchored near a known-good solution rather than free to wander indefinitely under noisy syndrome measurements — useful given that "all checks satisfied" alone does not guarantee the tracked weights are still correct (see the earlier discussion on degenerate syndrome solutions).

Measurement model

$$\mathbf{y}_i = h_i(\boldsymbol{\theta}_i) + \mathbf{v}_i, \quad R = \text{Cov}(\mathbf{v}_i), \quad \mathbf{y}_i = \mathbf{1}_M \quad (2)$$

The pseudo-label is the all-ones vector: the filter is driven to make every one of the M parity checks satisfied, which is the only supervision signal available. The measurement function h_i is the composition

$$h_i : \quad \boldsymbol{\theta}_i \xrightarrow[\mathbf{x}_i]{\text{ESCNN}} \tilde{\mathbf{L}}_i \rightarrow \mathbf{q}_i \rightarrow \mathbf{t}_i \rightarrow \mathbf{p}_i, \quad (3)$$

built from the network’s soft output and the parity-check matrix H (bit n , check m , entry H_{mn}):

$$\tilde{L}_{i,n} = f_{\theta_i}(\mathbf{X}_i) \quad (4)$$

$$q_{i,n} = \begin{cases} \tilde{L}_{i,n}, & n \text{ transmitted} \\ L_{\max}, & n \text{ filler} \\ 0, & n \text{ punctured} \end{cases} \quad (5)$$

$$t_{i,n} = \tanh\left(\frac{\text{clamp}(q_{i,n}, \pm L_{\max})}{2}\right) \quad (6)$$

$$p_{i,m} = \prod_{n: H_{mn}=1} t_{i,n}, \quad \mathbf{p}_i = [p_{i,1}, \dots, p_{i,M}]^T \quad (7)$$

Filler and punctured positions are fixed constants w.r.t. θ_i ; only the transmitted bits carry gradient information back to the weights. Noise covariances are taken isotropic, $Q = \sigma_Q^2 I$, $R = \sigma_R^2 I$.

Initialization

$$\hat{\theta}_{0|0} = \theta_{\text{pretrained}}, \quad P_{0|0} = \sigma_{P_0}^2 I, \quad Q = \sigma_Q^2 I, \quad R = \sigma_R^2 I \quad (8)$$

- $\hat{\theta}_{0|0}$ is set to the offline-pretrained ESCNN weights — the filter starts from a good point and only tracks drift away from it, rather than learning from scratch.
- $P_{0|0}$, Q , and R are all initialized as *scaled identity matrices*: a single scalar variance times I , rather than any structured (e.g. per-layer or per-check) covariance. This keeps the filter cheap to initialize and gives three interpretable knobs:
 - $\sigma_{P_0}^2$ — initial confidence in $\hat{\theta}_{0|0}$ (larger $\Rightarrow$ the first few updates are allowed to move the weights more).
 - σ_Q^2 — how fast the weights are allowed to drift per frame (tracking speed vs. stability trade-off).
 - σ_R^2 — how much the syndrome measurement is trusted (larger $\Rightarrow$ softer, more conservative updates).
 - α (only if the AR(1) dynamics model is used) — reversion strength toward $\theta_{\text{pretrained}}$; $\alpha \rightarrow 1$ behaves like the random walk, smaller α anchors the estimate more tightly.
- All three scalars are hyperparameters to be tuned (e.g. via a validation sweep, or by monitoring filter consistency through the normalized innovation $\Delta \mathbf{y}_i^T S_i^{-1} \Delta \mathbf{y}_i$, which should track $\chi^2(M)$ if Q and R are well-scaled). The scaled-identity choice is a starting point, not a constraint — it can later be replaced with a block-diagonal or per-layer form if needed.

1.2 Predict step

$$\hat{\theta}_{i|i-1} = f(\hat{\theta}_{i-1|i-1}) \quad (\text{propagate the estimate through the dynamics}) \quad (9)$$

$$F_i = \left. \frac{\partial f(\theta)}{\partial \theta} \right|_{\hat{\theta}_{i-1|i-1}} \quad \left(\Rightarrow \theta_i - \hat{\theta}_{i|i-1} \approx F_i(\theta_{i-1} - \hat{\theta}_{i-1|i-1}) + \mathbf{w}_i \right) \quad (10)$$

$$P_{i|i-1} = F_i P_{i-1|i-1} F_i^T + Q \quad (= \text{Cov}(\theta_i - \hat{\theta}_{i|i-1}): \text{old uncertainty} + \text{drift}) \quad (11)$$

Equation (10) comes from a first-order Taylor expansion, $f(\theta) \approx f(\hat{\theta}_{i-1|i-1}) + F_i(\theta - \hat{\theta}_{i-1|i-1})$, which makes the prediction error a linear function of the previous estimation error plus the process noise. Since $\mathbf{w}_i$ is independent of that error, its covariance is (11). For both dynamics models used here f is affine, so the expansion is exact.

What needs to be computed:

1. Propagate the weight estimate through f .
 - Random walk: $\hat{\boldsymbol{\theta}}_{i|i-1} = \hat{\boldsymbol{\theta}}_{i-1|i-1}$ (no-op).
 - AR(1): $\hat{\boldsymbol{\theta}}_{i|i-1} = \alpha \hat{\boldsymbol{\theta}}_{i-1|i-1} + (1 - \alpha) \boldsymbol{\theta}_{\text{pretrained}}$ — a single scalar-weighted average with the fixed pretrained checkpoint.
2. F_i , the Jacobian of the dynamics model. Both proposed choices of f are affine in $\boldsymbol{\theta}$, so F_i is a known constant, not something that needs autodiff: $F_i = I$ for the random walk, $F_i = \alpha I$ for AR(1). The predict covariance is then simply $P_{i|i-1} = \alpha^2 P_{i-1|i-1} + Q$ (random walk is the $\alpha = 1$ special case).

1.3 Update step

$$\hat{\mathbf{y}}_{i|i-1} = h_i(\hat{\boldsymbol{\theta}}_{i|i-1}) = \mathbf{p}_i \quad (\text{predicted checks}) \quad (12)$$

$$\Delta \mathbf{y}_i = \mathbf{1}_M - \mathbf{p}_i \quad (\text{innovation: unmet parity}) \quad (13)$$

$$H_i = \left. \frac{\partial h_i(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \right|_{\hat{\boldsymbol{\theta}}_{i|i-1}} \quad \left(\Rightarrow \Delta \mathbf{y}_i \approx H_i(\boldsymbol{\theta}_i - \hat{\boldsymbol{\theta}}_{i|i-1}) + \mathbf{v}_i \right) \quad (14)$$

$$S_i = H_i P_{i|i-1} H_i^T + R \quad (= \text{Cov}(\Delta \mathbf{y}_i)) \quad (15)$$

$$K_i = P_{i|i-1} H_i^T S_i^{-1} \quad (= \text{Cov}(\boldsymbol{\theta}_i, \Delta \mathbf{y}_i) \text{Cov}(\Delta \mathbf{y}_i)^{-1}) \quad (16)$$

$$\hat{\boldsymbol{\theta}}_{i|i} = \hat{\boldsymbol{\theta}}_{i|i-1} + K_i \Delta \mathbf{y}_i \quad (\text{LMMSE correction of } \hat{\boldsymbol{\theta}}_{i|i-1}) \quad (17)$$

$$P_{i|i} = P_{i|i-1} - K_i S_i K_i^T \quad (\text{uncertainty removed by } \Delta \mathbf{y}_i) \quad (18)$$

Equation (14) comes from a first-order Taylor expansion, $h_i(\boldsymbol{\theta}) \approx \mathbf{p}_i + H_i(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_{i|i-1})$, which makes the innovation a linear, noisy measurement of the weight error. Equations (15)–(18) are then the standard linear-Gaussian update: with $\text{Cov}(\boldsymbol{\theta}_i - \hat{\boldsymbol{\theta}}_{i|i-1}) = P_{i|i-1}$ and $\mathbf{v}_i \sim (\mathbf{0}, R)$, (15) and the cross-covariance $P_{i|i-1} H_i^T$ follow directly from the linearized model.

What needs to be computed:

1. *Forward pass*: run $\boldsymbol{\theta}_i \rightarrow \tilde{\mathbf{L}}_i \rightarrow \mathbf{q}_i \rightarrow \mathbf{t}_i \rightarrow \mathbf{p}_i$ once to get $\hat{\mathbf{y}}_{i|i-1}$ and the innovation $\Delta \mathbf{y}_i$.
2. *Jacobian H_i* : this does *not* need to be hand-derived. It is obtained automatically by differentiating the same forward pass with reverse-mode autodiff (e.g. `torch.func.jacrev` or `vjp` over h_i as a function of $\boldsymbol{\theta}$). Since the number of parity checks M is typically far smaller than the number of weights d , reverse mode is the efficient direction (cost $\sim M$ backward passes).
3. *Pruning*: any check row m that includes a punctured bit has $p_{i,m} = 0$ identically, and its corresponding row of H_i is also identically zero. These rows carry no information and should be dropped before forming S_i . (Refined further in Section 1.4: in practice almost every row touches a punctured bit, so this naive rule alone is not enough — punctured bits are first estimated from the code’s own structure, and pruning is then based on that estimation’s provenance, not simply on whether a row touches $\mathcal{P}$.)
4. *Gain and covariance update*: standard linear algebra once H_i (or its action on vectors, if d is too large to form H_i explicitly) is available. For large d , avoid materializing P densely — use a low-rank or diagonal approximation and matrix-free $H_i v / H_i^T v$ products instead.

1.4 The punctured-bit fallback and its self-referential failure mode

The pruning rule in Section 1.3 — drop any check row that touches a punctured bit — is a simplification that does not survive contact with real operating points. At MCS2, `num_res = 96` ($n_{\text{ldpc}} = 4576$, $M = 3696$), *every single check row* touches at least one punctured position, so that rule alone leaves $\mathbf{y}_i$ empty and the update step vacuous. What is actually implemented is a two-stage scheme: first estimate the punctured bits from the code’s own structure, then prune more carefully than “touches a punctured bit” — this section derives why the second step is necessary.

Erasure-peeling estimate of the punctured bits

Let $\mathcal{P} \subset \{1, \dots, N\}$ be the punctured bit indices (where $q_{i,n} = 0$ by construction, hence $t_{i,n} = 0$). For $n \in \mathcal{P}$ still unresolved ($t_{i,n} = 0$), a check m with $H_{mn} = 1$ can send n a message only if n is check m 's *sole* remaining unresolved neighbour:

$$\mu_{m \rightarrow n} = 2 \operatorname{artanh} \left(\prod_{\substack{n' \neq n \\ H_{mn'} = 1}} t_{i,n'} \right), \quad \text{sent iff } |\{n' \in \mathcal{P} : H_{mn'} = 1, t_{i,n'} = 0\}| = 1. \quad (19)$$

Each punctured bit's estimate sums every message it receives this round, $L_n = \sum_{m \in \mathcal{C}(n)} \mu_{m \rightarrow n}$, $t_{i,n} \leftarrow \tanh(L_n/2)$, where $\mathcal{C}(n)$ is the (possibly empty) set of checks that contributed to n this round. Resolving one bit can unlock a check for its neighbours next round, so this is iterated for a fixed round budget (`tsyn_fallback_iters`), stopping early at a fixpoint.

The circularity this introduces

Define $\text{contributing}(m) = 1$ if check m ever supplied a message to one of its own neighbours, i.e. $m \in \mathcal{C}(n)$ for some n . If check m is later scored using that same n 's estimate, it is tested against a value it helped fabricate. Note that this definition — and the exclusion rule built on it below — is entirely degree-agnostic: *a check is excluded if it ever contributed a message to resolve one of its own punctured neighbours, regardless of whether that neighbour's degree is 1 or 22*. Degree only determines how *likely* a check is to end up excluded (see below), not whether the exclusion criterion itself applies.

This is not merely inelegant — it is provably vacuous in the worst case. Suppose n has degree 1 (check m is the *only* row touching it) and gets resolved. Then $\mathcal{C}(n) = \{m\}$ necessarily, and writing $X := \prod_{n' \neq n, H_{mn'} = 1} t_{i,n'}$,

$$t_{i,n} = \tanh(\operatorname{artanh}(X)) = X \implies p_{i,m} = t_{i,n} \cdot X = X^2 \geq 0 \text{ identically.} \quad (20)$$

$p_{i,m}$ depends only on $|X|$ — the *magnitude* of check m 's real, transmitted neighbours — and is completely blind to their sign. Flipping the sign of any one real neighbour (exactly the event a parity check exists to catch) leaves $p_{i,m}$ unchanged. Such a check rewards confident outputs regardless of correctness; scored into the EKF's innovation, it biases $\hat{\theta}_i|_i$ toward sharpening the network's confidence rather than correcting its errors. For n with degree > 1 , resolved from several checks at once, the same leakage is present but diluted — $t_{i,n}$ mixes m 's own contribution with genuinely extrinsic information from the other resolving checks.

Classification of checks: clean / self-referential / dead

Run the peeling process to its round budget or fixpoint, and classify every check row by the result:

- **dead**: at least one neighbour of m is punctured and never resolved ($t_{i,n} = 0$ for some $n \in \mathcal{P} \cap \{n : H_{mn} = 1\}$). Since $t_{i,n} = 0$ identically and does not depend on θ_i , $p_{i,m} = 0$ identically too — the corresponding row of H_i is exactly zero, contributing nothing to the update (harmless, but wasted compute if left in).
- **self-referential**: every neighbour of m is resolved, and $\text{contributing}(m) = 1$ — m helped produce at least one of the values it would be tested against.
- **clean**: every neighbour of m is resolved, and $\text{contributing}(m) = 0$ — m only *consumes* estimates that came entirely from other checks.

Crucially, a degree-1 punctured bit can never yield a clean check: its only possible contributor and its only possible consumer are the same row, so if it resolves at all, that row is self-referential by construction. Degree-1 bits are not the only source of self-referential rows, though — for a bit resolved from several checks (e.g. degree 22 at MCS2), the checks that did the resolving are self-referential even though the bit's value itself is legitimately multi-sourced; other checks that merely consume that value, without having contributed to it, remain clean.

This classification depends only on H and which positions are punctured — never on θ_i or the frame index i , since punctured positions are always exactly 0 at initialization regardless of the forward pass. It is therefore computed once, not per update.

Example (MCS2, num_res = 96; $Z = 88$, $n_{\text{ldpc}} = 4576$). Three real rows illustrate all three outcomes:

- *Dead*: bit 3080 (degree 1) is check 2200’s only unresolved neighbour candidate, but check 2200 also touches bit 72 (punctured, still unresolved that round) — two simultaneous unknowns block the update rule, so 3080 never receives a value.
- *Self-referential*: bit 4136 (degree 1) is resolved solely by check 3256 (real neighbours 959, 1182). With $t_{959} = 0.9$, $t_{1182} = -0.7$: $t_{4136} = -0.63$ and $p_{3256} = (-0.63)(0.9)(-0.7) = 0.3969$. Flipping $t_{1182} \rightarrow +0.7$ gives $t_{4136} = 0.63$ and $p_{3256} = 0.3969$ again — unchanged, exactly as predicted by $p = X^2$.
- *Clean*: bit 17 (degree 22) is resolved from six independent checks (88, 586, 982, 1376, 1604, 1885), none of which is check 371. Check 371 (neighbours 17, 174, 982, 1251) never contributed to either of its two punctured neighbours, so its score genuinely reflects whether four independently-sourced values are jointly consistent.

At this operating point: 704/3696 rows clean, 1776/3696 self-referential, 1216/3696 dead.

Exclusion rule (supersedes the Section 1.3 pruning step)

Only clean rows are scored:

$$\mathcal{M}_{\text{clean}} = \{ m : \text{every neighbour of } m \text{ resolved, and } \text{contributing}(m) = 0 \}, \quad \mathbf{y}_i, \mathbf{p}_i, H_i \text{ restricted to } \mathcal{M}_{\text{clean}}. \quad (21)$$

Dead rows are already harmless (zero row in H_i) and self-referential rows are actively biased, not merely uninformative, so both are dropped — refining “drop rows touching a punctured bit” in Section 1.3 into “drop rows that are dead *or* self-referential.” Since $\mathcal{M}_{\text{clean}}$ is static, this reduces to a fixed row-index mask computed once from H , at negligible runtime cost per update.

Remark: raising the round budget does not help. Increasing `tsyn_fallback_iters` beyond 1 round does *not* grow $\mathcal{M}_{\text{clean}}$ — empirically (MCS2, MCS6, MCS12, MCS16; rounds 1 through 20) $|\mathcal{M}_{\text{clean}}|$ is identical at every round count, while dead rows convert entirely into self-referential ones by round 2–3. The reason is structural: a row m becomes clean only if *all* of its punctured neighbours resolve simultaneously, via other checks, in the very round before m would otherwise have become their sole remaining resolver. That either happens on the first round in which m ’s unresolved count would drop to zero, or it never happens — once m itself becomes the sole remaining resolver for one of its neighbours, at any round, m is self-referential from then on, permanently. More rounds only decide *when* a still-dead row’s fate resolves, never whether it resolves clean.

2 Supervised EKF with known calibration bits (ekfi)

`ekfi` is the supervised counterpart of `ekf`, and the EKF counterpart of the SGD genie baseline `sgdbcei` (the “i” stands for *impractical*). It uses the same tracker (`EkfParamTracker`), the same state model and the same `escnn_ekf_*` hyperparameters as Section 1; only the measurement, and the slots it is taken on, change.

2.1 Calibration region

For every group of $S_{\text{data}} = \text{slots_per_group}$ scored data slots, the transmitter sends an additional, separate region of $S_{\text{cal}} = \text{calib_slots_per_group}$ calibration slots (`_build_calib_training_data` in `ekf.py`):

- same structure as the data slots: 2 DMRS symbols + 12 payload symbols of fresh, CRC+LDPC-coded random bits;

- same channel realization $\mathbf{h}$ as the group’s data, and the same random draw as `sgdbcei`, so the two modes see identical calibration data;
- its own DMRS channel estimate and its own LMMSE prior, which is fed to the ESCNN as the augmentation input;
- used only for the EKF updates and never scored. The scored data slots are never measured in this mode.

The calibration payload bits $b_{s,n} \in \{0, 1\}$ are assumed known at the receiver; this is what makes the mode a genie baseline rather than a deployable scheme.

2.2 Measurement model

For calibration slot s and user k , let $\tilde{L}_{s,n}(\boldsymbol{\theta})$, $n = 1, \dots, N$, be the ESCNN output LLRs of that slot (convention $\tilde{L} > 0 \Leftrightarrow b = 1$), with $N = 12 q_m B$ payload bits per slot ($B = \text{num_res}$; e.g. $N = 4608$ for MCS39, 16QAM, at $B = 96$). Each bit gives one measurement, its *agreement* with the known bit:

$$a_{s,n}(\boldsymbol{\theta}) = (2b_{s,n} - 1) \tanh(\tilde{L}_{s,n}(\boldsymbol{\theta})/2) \in [-1, 1] \quad (+1: \text{confidently correct}, -1: \text{confidently wrong}) \quad (22)$$

$$\mathbf{y}_s = \mathbf{1}_N \quad (\text{target: every bit confidently correct}) \quad (23)$$

$$\Delta \mathbf{y}_s = \mathbf{1}_N - \mathbf{a}_s, \quad \Delta y_{s,n} = 2 |b_{s,n} - \sigma(\tilde{L}_{s,n})| \quad (\text{innovation}) \quad (24)$$

$$\frac{\partial a_{s,n}}{\partial \boldsymbol{\theta}} = (2b_{s,n} - 1) \frac{1}{2} (1 - t_{s,n}^2) \frac{\partial \tilde{L}_{s,n}}{\partial \boldsymbol{\theta}} \quad (\text{row } n \text{ of } H_s; t_{s,n} = \tanh(\tilde{L}_{s,n}/2)) \quad (25)$$

The measurement has exactly the “target $\mathbf{1}$, innovation $\mathbf{1}$ –measurement” form of (13), so the tracker code is shared unchanged: (14)–(18) apply with $h_i \rightarrow \mathbf{a}_s$, $M \rightarrow N$, and $R = \sigma_R^2 I_N$. Unlike (7), each row of H_s involves a single network output: because $b_{s,n}$ is known, every bit can be checked on its own, whereas a parity check only constrains a combination of bits, so each p_m is a product over all d_c LLRs of the check. The row is still dense in $\boldsymbol{\theta}$, since $\tilde{L}_{s,n}$ depends on all shared convolution weights. In (7), the gradient reaching bit n is scaled by $\prod_{n' \neq n} t_{n'}$, so one uncertain bit ($t_{n'} \approx 0$) in a check silences that row for all of its bits; in (25) there is no such coupling. No clamp, filler or puncturing handling is needed either: the measured bits are exactly the transmitted ones.

2.3 Schedule

Per group: one predict step (9)–(11), then S_{cal} sequential update steps, one per calibration slot, with no predict in between. Only then is the updated network run on the group’s scored data slots. In `ekf`, the group also gets one predict, but the S_{data} updates are taken on the scored slots themselves, which are then scored with the updated weights (test-time adaptation).

2.4 What one update optimizes

An EKF update is one Gauss–Newton step, started at the prior mean, on the MAP objective

$$J(\boldsymbol{\theta}) = \frac{1}{\sigma_R^2} \|\mathbf{1} - h(\boldsymbol{\theta})\|^2 + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_{i|i-1})^T P_{i|i-1}^{-1} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_{i|i-1}). \quad (26)$$

For `ekfi` the data term is $\frac{4}{\sigma_R^2} \sum_n (b_n - \sigma(\tilde{L}_n))^2$, a Brier (squared probability error) loss, not the BCE used by `sgdbcei`. One practical consequence: for a *confidently wrong* bit, $\Delta y_n \approx 2$ but the factor $1 - t_n^2$ in (25) is ≈ 0 , so the EKF barely moves it, whereas the BCE gradient $\sigma(\tilde{L}_n) - b_n$ stays $\approx \pm 1$. The same factor appears in every $\partial p_m / \partial \boldsymbol{\theta}$ of `ekf`.

2.5 Comparison: ekf vs. ekfi

Numbers are for MCS39 (16QAM, $R = 0.21$) with $B = 96$ and 12 payload symbols per slot: $N = 12 \cdot 4 \cdot 96 = 4608$ coded bits, $k = 967$ information bits + 16 CRC = 983, which gives base graph 2 with lifting size

$Z = 104$ ($k_{\text{ldpc}} = 1040$, 57 filler bits, $n_{\text{ldpc}} = 5408$) and $42Z = 4368$ parity checks. The clean subset $\mathcal{M}_{\text{clean}} \subset \{1, \dots, 4368\}$ is computed from the parity-check matrix at start-up and printed as **fallback check quality: X/4368 clean**.

	ekf (Section 1)	ekfi (this section)
Same in both		
State, dynamics f , F_i	ESCNN weights left unfrozen by <code>escnn_load_freeze</code> ; random walk or AR(1)	
$P_{0 0}$, Q , R , α	Same <code>escnn_ekf_*</code> values, same scaled-identity form	
Predict / update equations	(9)–(11) and (14)–(18), same tracker code	
Innovation form	Target 1 , innovation $\mathbf{1} - h(\hat{\boldsymbol{\theta}}) \in [0, 2]$ per row	
Predicts per group	One	
Jacobian	<code>torch.func.jacrev</code> , chunked by <code>escnn_ekf_jacobian_chunk_size</code>	
Different		
Supervision	None: code parity only	Known transmitted bits (genie)
Slots measured	The scored data slots themselves	A separate calibration region, never scored
Extra transmission	None	S_{cal} calibration slots per group
Updates per group	$S_{\text{data}} = \text{slots_per_group}$	$S_{\text{cal}} = \text{calib.slots_per_group}$
Measurement row	Soft check $p_m = \prod_{n \in \mathcal{N}(m)} t_n$	Bit agreement $a_n = (2b_n - 1) t_n$
Rows per slot (per user)	$ \mathcal{M}_{\text{clean}} $, a subset of the 4368 checks	$N = 12 q_m B = 4608$
Network outputs per row	d_c (all bits of the check); gradient to each scaled by the others' t	One (the bit itself); no coupling
Sign information	Parity only: flipping an even number of bits in a check leaves p_m unchanged	Direct: each row says which way the bit should go
Puncturing / filler	Erasure peeling + static clean-row mask (Section 1.4)	Not applicable
LLR clamp	± 30 before tanh	None
Implicit data term	$\sum_m (1 - p_m)^2$	$4 \sum_n (b_n - \sigma(\tilde{L}_n))^2$ (Brier)
Meaning of σ_R^2	Noise on a check product	Noise on a bit agreement; same value, different quantity
Size of S	$ \mathcal{M}_{\text{clean}} \times \mathcal{M}_{\text{clean}} $, at most 4368×4368	4608×4608
ESCNN prior input	<code>which_augment</code> prior of the data slots	Always the LMMSE prior of the calibration slots
Main failure mode	Converging to confident, parity-consistent but wrong outputs	Limited by S_{cal} ; confident errors are hard to undo

Two of these differences deserve a note. First, σ_R is shared, but it scales measurements of different kinds and dimensions, so the same value does not give the two modes the same measurement trust; tuning it separately per mode would make the comparison fairer. Second, the calibration prior is always LMMSE, even when `which_augment` is, e.g., `AUGMENT_SPHERE`; in that case `ekfi` adapts the weights on a different prior from the one used at scoring time.

3 The `cos_true` diagnostic in `ekf`

`ekf` can log, for every update, how well the step it takes agrees with the step that knowledge of the true bits would have produced. This is a diagnostic only: it never changes the filter's trajectory.

3.1 What “true” is

When logging is on (`log_train_every_epochs > 0`), `ekf.py` passes `tx_ref`, the true CRC+LDPC-coded payload bits $b_{s,n}$ of the *same scored data slots* that `ekf` is updating on. It is not the calibration region of Section 2: those slots do not exist in `ekf` mode. The bits are the very ones the slot is later scored against.

3.2 How the reference step is computed

For each data slot s , inside the same update call and from the same prior $(\hat{\theta}_{i|i-1}, P_{i|i-1})$ the applied step uses, i.e. after the group’s predict and after the syndrome updates of slots $1, \dots, s-1$:

$$\mathbf{r}_s(\theta) = [(2b_{s,n} - 1) \tanh(\tilde{L}_{s,n}(\theta)/2)]_{n=1}^N \quad (\text{agreement (22), on slot } s \text{ itself}) \quad (27)$$

$$H_s^{\text{ref}} = \left. \frac{\partial \mathbf{r}_s}{\partial \theta} \right|_{\hat{\theta}_{i|i-1}}, \quad S_s^{\text{ref}} = H_s^{\text{ref}} P_{i|i-1} H_s^{\text{ref}T} + R \quad (\text{same } P_{i|i-1}, \text{ same } \sigma_R^2) \quad (28)$$

$$\Delta \theta_s^{\text{ref}} = P_{i|i-1} H_s^{\text{ref}T} (S_s^{\text{ref}})^{-1} (\mathbf{1}_N - \mathbf{r}_s(\hat{\theta}_{i|i-1})) \quad (\text{computed, never applied}) \quad (29)$$

$$\Delta \theta_s^{\text{syn}} = K_i \Delta \mathbf{y}_i \quad (\text{the step actually applied, (17)}) \quad (30)$$

$$\text{cos_true} = \frac{\langle \Delta \theta_s^{\text{syn}}, \Delta \theta_s^{\text{ref}} \rangle}{\|\Delta \theta_s^{\text{syn}}\| \|\Delta \theta_s^{\text{ref}}\|} \quad (+1 \text{ aligned, } 0 \text{ unrelated, } < 0 \text{ opposed}) \quad (31)$$

The LLRs are the same ones that form the syndrome measurement: same network input (received samples plus prior), same real-bit selection, and the same $N = 12 q_m B$ positions, which equal the LDPC codeword length `helper.n`. The cost is a second Jacobian with N rows and an $N \times N$ solve per slot.

`cos_true` is therefore a one-step counterfactual: from exactly where the `ekf` filter is now, would a genie that knows this slot’s bits push the weights the same way? It does not track an `ekfi` trajectory, and it compares directions only, not step sizes.

3.3 Comparison: applied ekf step vs. the cos_true reference

	<code>ekf</code> applied step	<code>cos_true</code> reference	<code>ekfi</code> step (Sec. 2)
Slots measured	Scored data slot s	Same scored data slot s	Calibration slot, never scored
Ground truth	None	True bits of slot s	True bits of the calibration slot
Measurement	Soft checks p_m , clean rows	Bit agreement r_n	Bit agreement a_n
Rows	$ \mathcal{M}_{\text{clean}} $	N	N
LLR clamp / puncturing	± 30 ; peeling + clean mask	None; not needed	None; not needed
Prior $(\hat{\theta}, P)$	<code>ekf</code> ’s own	Identical to the applied step	<code>ekfi</code> ’s own trajectory
R	$\sigma_R^2 I$	$\sigma_R^2 I$ (same value)	$\sigma_R^2 I$ (same value)
Applied to weights	Yes	No	Yes
Purpose	Tracking	Diagnostic: is the blind step pointed the right way?	Genie upper-bound baseline

Reading the diagnostic.

- The reference is stronger than `ekfi`: it knows the bits of the slot being scored, whereas `ekfi` only knows calibration bits from the same channel.
- The cosine is Euclidean over all tracked weights, so the parameter groups with the largest step components (e.g. the element-wise scales vs. the convolution taps) dominate it. A per-group cosine, computed

separately for the scale vector and for each convolution layer, would show which part of the network the blind step gets right.

- Logging the norm ratio $\|\Delta\theta^{\text{syn}}\|/\|\Delta\theta^{\text{ref}}\|$ alongside the cosine would separate “right direction, too small” from “wrong direction”.